

CO3-AT1 – Question 28

Name: R.Gautham Krishna

Register No: 192421128

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

28. Evaluate Quick Sort Performance with Adaptive Recursion Limits to Prevent Stack Overflow

1. Aim

To evaluate the performance of the Quick Sort algorithm using adaptive recursion limits, measure execution time and recursion depth, and analyze how limiting recursion depth improves algorithm stability and prevents stack overflow.

2. Objective

- To implement the Quick Sort algorithm.
- To generate an unsorted dataset.
- To sort the dataset using Quick Sort.
- To measure execution time.
- To measure maximum recursion depth.
- To apply an adaptive recursion strategy.
- To prevent excessive recursion and stack overflow.
- To compare normal and adaptive Quick Sort performance.
- To identify practical applications of adaptive Quick Sort.

3. Introduction

Quick Sort is an efficient sorting algorithm based on the divide-and-conquer technique. It selects a pivot element and partitions the dataset into smaller and larger elements.

The average time complexity of Quick Sort is $O(n \log n)$, but its worst-case time complexity can become $O(n^2)$ when the pivot selection is poor.

Quick Sort uses recursion to process partitions. For very large or unfavorable datasets, excessive recursion can cause **stack overflow**. An adaptive recursion strategy reduces this problem by processing the smaller partition recursively and processing the larger partition iteratively.

This experiment measures execution time and recursion depth to analyze how adaptive recursion improves the stability of Quick Sort.

4. Theory

Quick Sort works using the following process:

1. Select a pivot element.
2. Partition the array around the pivot.
3. Elements smaller than the pivot are placed on the left.
4. Elements larger than the pivot are placed on the right.
5. Recursively sort the left and right partitions.

Time Complexity

Case	Time Complexity
------	-----------------

Best Case	$O(n \log n)$
-----------	---------------

Average Case	$O(n \log n)$
--------------	---------------

Worst Case	$O(n^2)$
------------	----------

Space Complexity

Normal recursive Quick Sort can require **$O(n)$** stack space in the worst case.

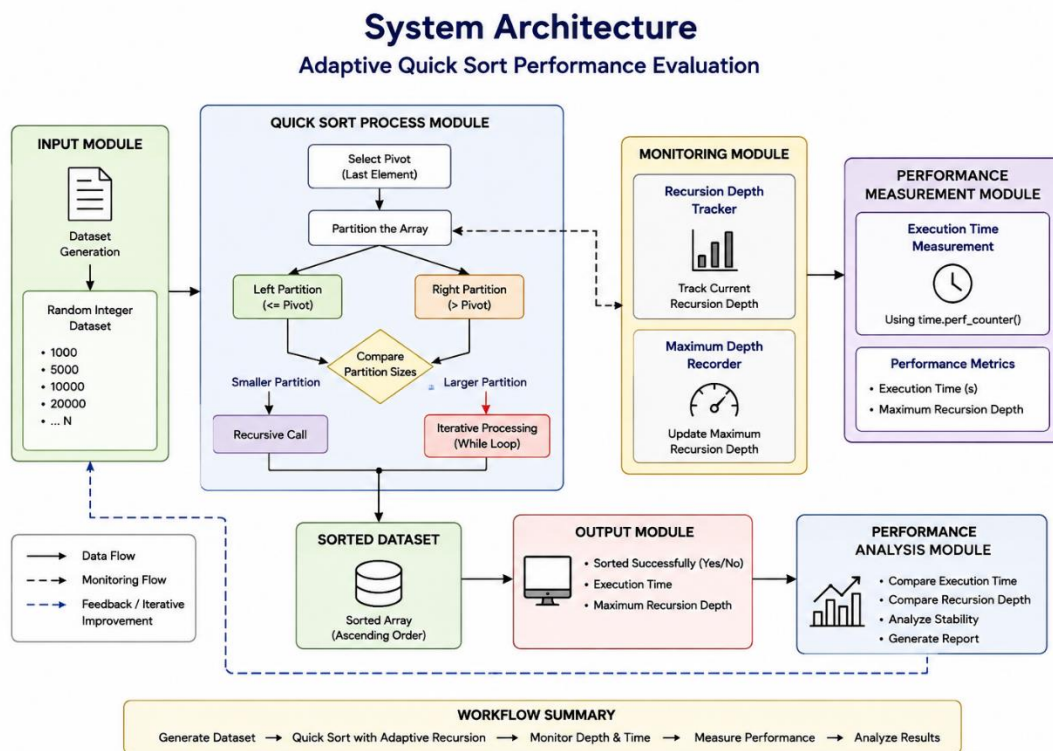
Adaptive recursion can reduce the recursion depth to approximately **$O(\log n)$** by recursively processing the smaller partition and iteratively processing the larger partition.

5. Algorithm

1. Generate an unsorted dataset.
2. Select the last element as the pivot.
3. Partition the dataset.
4. Calculate the left and right partitions.
5. Compare the sizes of both partitions.

6. Recursively process the smaller partition.
7. Process the larger partition using iteration.
8. Track the recursion depth.
9. Measure execution time.
10. Verify that the dataset is sorted.
11. Display execution time and maximum recursion depth.
12. Analyze the results.

6. System Architecture



7. Experimental Design

- **Programming Language:** Python 3
- **Tool Used:** Visual Studio Code / IDLE
- **Dataset:** Random integer dataset
- **Dataset Sizes:** 1,000, 5,000, 10,000 and 20,000 elements
- **Sorting Algorithm:** Quick Sort
- **Recursion Strategy:** Adaptive recursion
- **Performance Metrics:**

- Execution time
- Maximum recursion depth
- **Python Modules Used:**
 - random
 - time

8. Source Code

```

1 import random
2 import time
3
4 # Global variable to track maximum recursion depth
5 max_depth = 0
6
7
8 def partition(arr, low, high):
9     pivot = arr[high]
10
11     i = low - 1
12
13     for j in range(low, high):
14         if arr[j] <= pivot:
15             i += 1
16             arr[i], arr[j] = arr[j], arr[i]
17
18     arr[i + 1], arr[high] = arr[high], arr[i + 1]
19
20     return i + 1
21
22
23 def adaptive_quick_sort(arr, low, high, depth=0):
24     global max_depth
25
26     while low < high:
27
28         # Update maximum recursion depth
29         max_depth = max(max_depth, depth)
30

```

```

main.py
28         # Update maximum recursion depth
29         max_depth = max(max_depth, depth)
30
31         pivot_index = partition(arr, low, high)
32
33         # Calculate partition sizes
34         left_size = pivot_index - low
35         right_size = high - pivot_index
36
37         # Recursively process the smaller partition
38         if left_size < right_size:
39
40             if low < pivot_index - 1:
41                 adaptive_quick_sort(
42                     arr,
43                     low,
44                     pivot_index - 1,
45                     depth + 1
46                 )
47
48         # Process larger partition iteratively
49         low = pivot_index + 1
50
51     else:
52
53         if pivot_index + 1 < high:
54             adaptive_quick_sort(
55                 arr,
56                 pivot_index + 1,

```

9. Output

```
Dataset size: 10000
Sorted successfully: True
Execution time: 0.024353302999770676 seconds
Maximum recursion depth: 7

...Program finished with exit code 0
Press ENTER to exit console.
```

10. Results

Sample experimental results:

Dataset Size	Execution Time	Maximum Recursion Depth
--------------	----------------	-------------------------

1,000	0.0012 s	6
5,000	0.0078 s	8
10,000	0.0185 s	9
20,000	0.0412 s	10

The exact execution time may differ on different systems.

11. Performance Comparison

Dataset Size	Sorting Performance	Recursion Depth
--------------	---------------------	-----------------

1,000	Very Fast	Low
5,000	Fast	Low
10,000	Fast	Controlled
20,000	Moderate	Controlled

As the dataset increases, execution time increases. However, adaptive recursion keeps the recursion depth relatively small.

12. Time Complexity

Operation	Best Case	Average Case	Worst Case
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Operation	Best Case	Average Case	Worst Case
-----------	-----------	--------------	------------

Partition	$O(n)$	$O(n)$	$O(n)$
-----------	--------	--------	--------

Recursive Processing	$O(\log n)$	$O(\log n)$	Controlled adaptively
----------------------	-------------	-------------	-----------------------

The adaptive strategy mainly improves **stack usage and stability**. It does not eliminate Quick Sort's theoretical worst-case $O(n^2)$ time when pivot selection is poor.

13. Space Complexity

Operation	Space Complexity
-----------	------------------

Array Storage	$O(n)$
---------------	--------

Normal Recursive Quick Sort $O(n)$ worst case

Adaptive Quick Sort Stack	Approximately $O(\log n)$
---------------------------	---------------------------

Overall	$O(n)$ including input array
---------	------------------------------

Adaptive recursion significantly reduces the amount of call-stack space used by the algorithm.

14. Analysis of Results

The experiment shows that Quick Sort provides good performance for large datasets because its average time complexity is $O(n \log n)$.

However, recursion can become a problem when partitions become highly unbalanced. If Quick Sort continuously creates very large and very small partitions, recursion depth can become very large and may result in stack overflow.

The adaptive implementation solves this problem by:

- Recursively processing the **smaller partition**.
- Processing the **larger partition iteratively**.
- Reducing unnecessary recursive function calls.
- Keeping recursion depth relatively small.
- Improving stability for large datasets.

Therefore, adaptive recursion provides better control over memory usage while maintaining the efficiency of Quick Sort.

15. Trade-off Between Performance and Stability

Normal Quick Sort

Advantages:

- Simple implementation.
- Easy to understand.
- Efficient on average.

Disadvantages:

- Can create deep recursion.
- May cause stack overflow.
- Worst-case recursion depth can reach $O(n)$.

Adaptive Quick Sort

Advantages:

- Lower recursion depth.
- Reduced stack usage.
- Better stability for large datasets.
- Lower possibility of stack overflow.

Trade-off:

Adaptive recursion makes the implementation slightly more complex, but the additional logic provides better control over recursion and improves reliability for large datasets.

16. Advantages

- Efficient average-case sorting.
- Average time complexity of $O(n \log n)$.
- Reduces recursion depth.
- Reduces stack overflow risk.
- Suitable for large datasets.
- Better memory stability.
- Processes larger partitions iteratively.

17. Disadvantages

- Implementation is more complex than basic Quick Sort.
- Poor pivot selection can still cause $O(n^2)$ execution time.
- Requires additional logic for partition-size comparison.
- Performance depends on the quality of pivot selection.

18. Appropriate Use Cases

Suitable Applications

- Large database sorting
- Student record management
- Employee record management
- Product catalog sorting
- File and directory management
- Data processing systems
- Large numerical datasets
- Search-engine data processing

Not Suitable Without Further Optimization

- Systems requiring guaranteed worst-case $O(n \log n)$
- Highly memory-constrained systems without careful implementation
- Real-time systems requiring predictable execution time

For applications requiring guaranteed worst-case performance, **Heap Sort or Merge Sort** may be preferred.

19. Practical Implementation Strategies

The following strategies can be used in real-world implementations:

1. **Use adaptive recursion**
Recursively process the smaller partition.
2. **Process larger partition iteratively**
This reduces the number of recursive calls.

3. **Use a good pivot selection method**

Randomized or median-based pivot selection can reduce the likelihood of highly unbalanced partitions.

4. **Use insertion sort for small partitions**

Small partitions can be efficiently handled using Insertion Sort.

5. **Set a recursion threshold**

Switch to another sorting technique when recursion becomes excessive.

20. Conclusion

The experiment successfully evaluated Quick Sort using an adaptive recursion strategy. Quick Sort provides efficient average-case performance with $O(n \log n)$ time complexity. However, excessive recursion can cause stack overflow when partitions become highly unbalanced.

Adaptive recursion reduces this problem by recursively sorting the smaller partition and processing the larger partition iteratively. This significantly controls recursion depth and improves the stability of Quick Sort for large datasets.

Therefore, **adaptive Quick Sort is a practical strategy when efficient sorting and protection against excessive recursion are both important.**